Module 2 — Workflows & Best Practices

Lesson 2.1

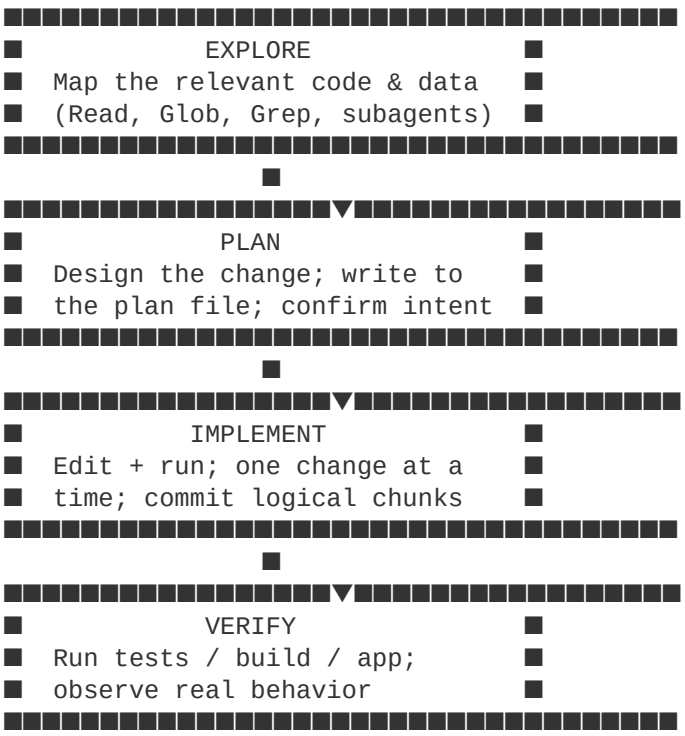
The planning loop

Explore → Plan → Implement → Verify. The single workflow that handles 90% of non-trivial tasks well.

Why it matters

Jumping from request to implementation is the single biggest source of bad Claude-assisted code. Adding two cheap steps before — exploration and planning — converts "write code and pray" into a process you can debug.

The loop



Step 1 — Explore (cheap, fast)

Goal: build enough mental model to plan accurately. Do not implement.

Patterns:

> *"Map the auth flow end-to-end. Identify the entry point, the middleware, the session store, and where logout invalidates state."*
>
> *"Use the Explore subagent to find every place that constructs a* `User` *object. Report the top 10 hot spots."*

Don't skip this. A 30-second exploration prevents a 30-minute rewrite.

Step 2 — Plan (in plan mode if non-trivial)

Goal: get explicit about *what* you'll change *where* and *why*.

For small tasks (1 file, <50 lines), the plan can be a sentence in the chat.

For everything else, use plan mode (see 1.4). The plan should answer:

- What's the smallest set of files that need to change?

- What existing code should be reused (not re-built)?
- What's the verification step?

If the plan says "refactor X for cleanliness" — push back. Cleanliness isn't a goal in this loop. Behavior change is.

Step 3 — Implement (small, reversible chunks)

Goal: get to a working state quickly, then iterate.

- Make the smallest change that proves the approach works. (Add the endpoint with hardcoded data; wire DB later.)
- Commit logical chunks as you go. Don't accumulate 500 lines of uncommitted changes.
- Avoid scope creep. If you notice unrelated issues, write them down and keep moving.

Step 4 — Verify (the most-skipped step)

Goal: confirm the change actually does what you claimed.

Levels:

Level	What
Type check	`tsc`, `mypy`, `cargo check` — fast, narrow
Tests	unit + integration — what's automated catches what's repeatable
Manual	actually run the app and exercise the change
Observability	check logs / metrics in a sandbox

The `/verify` skill (in supported versions) automates step 3. Use it.

Type checks pass ≠ the feature works. A common Claude failure mode is "all green, ship it" when the code is type-safe but logically wrong. Always do at least one manual exercise of any user-facing change.

Common deviations and why they fail

Deviation	Failure mode
Skip explore	Implementation conflicts with existing patterns; rework
Skip plan	Mid-implementation re-architecting; thrash
Skip verify	"Worked on my machine" — bugs land in PR
All four in one prompt	Vague request → vague plan → vague implementation

Try it

Pick a small backlog item. Force yourself to:

1. Spend a 5-minute explore.
2. Write a plan (plan mode).
3. Implement only what the plan says.
4. Verify by running the app, not just tests.

Notice where you were tempted to skip. That's where bugs live.

Gotchas

- **Plans rot.** If exploration revealed something new mid-implementation, update the plan rather than silently deviating.
- **Verification has to match the user's reality.** Headless test pass ≠ in-browser correctness for a UI change. Open the browser.
- **The loop is not waterfall.** You'll bounce back to explore mid-implement when surprised. Expected.

Further reading

- 1.4 Plan mode
- 2.4 Debugging patterns — when the loop gets ugly

Next

→ 2.2 TDD with Claude